

El Modelo de Especialización de Roles (RSM): Coordinación de Herramientas Basadas en LLM en el Desarrollo de Software Agéntico – Un Estudio de Caso Exploratorio

Carlos Alberto Fernández-y-Fernández ^{1*}, Jorge R. Aguilar Cisneros ²

¹ Instituto de Computación, Universidad Tecnológica de la Mixteca, México

² Decanato de Ingenierías, UPAEP, Puebla, México

*e-mail: caff@mixteco.utm.mx

RESUMEN

La integración de modelos de lenguaje de gran tamaño (LLM) en los flujos de trabajo de desarrollo de software ha dado lugar a un paradigma conocido como Ingeniería de Software Agéntica (SE 3.0), en el cual agentes autónomos gestionan ciclos de vida de desarrollo completos bajo supervisión humana. Este artículo presenta un estudio de caso exploratorio en el que tres herramientas basadas en LLM—Antigravity (un IDE agéntico con backend Gemini 2.5), Gemini CLI y Qwen Code (ejecución local mediante Ollama)—se coordinan según un marco de distribución de roles propuesto en este trabajo como el Modelo de Especialización de Roles (RSM). Tres preguntas de investigación guían el estudio: (RQ1) cómo se pueden coordinar herramientas basadas en LLM con capacidades distintas a través del RSM en un flujo de trabajo de desarrollo real; (RQ2) qué desviaciones de la distribución de roles planificada emergen durante la ejecución del RSM y qué factores las explican; y (RQ3) cómo se compara el producto resultante con el modelo de calidad ISO/IEC 25010. El objetivo fue proponer el RSM mediante el desarrollo incremental de una aplicación de escritorio en Python para la visualización interactiva de datos climáticos. Se documentan el flujo de trabajo, las desviaciones observadas, las técnicas de blindaje de prompts (prompt-hardening) y una evaluación cualitativa de la calidad. Los resultados sugieren que la coordinación explícita de roles puede respaldar la organización del ciclo de desarrollo y la calidad arquitectónica, pero requiere estrategias de coordinación deliberadas, gestión de contexto y verificación humana de los resultados generados por los agentes.

Palabras clave: ingeniería de software agéntica; SE 3.0; LLM; generación de código; especialización de roles; orquestación multi-agente; vibe coding; ingeniería de prompts.

1. Introducción

El campo de la ingeniería de software está experimentando una transformación paradigmática impulsada por la integración de modelos de lenguaje de gran tamaño (LLM) como agentes autónomos capaces de gestionar ciclos de vida de desarrollo completos. Esta evolución se ha conceptualizado como Ingeniería de Software Agéntica, o SE 3.0, un marco en el que el desarrollador humano actúa como arquitecto, mentor y evaluador de agentes inteligentes en lugar de como productor directo de código [1].

En este contexto, ha emergido el concepto de *vibe coding*, un término popularizado por Andrej Karpathy en febrero de 2025 [2] para describir una práctica de desarrollo en la que el programador se comunica con el modelo de forma conversacional, priorizando el resultado percibido sobre la comprensión estructural del código generado [3]. Si bien esta práctica democratiza el acceso al desarrollo de software, plantea interrogantes sobre la confiabilidad arquitectónica del código producido y la erosión a largo plazo de las competencias técnicas profundas de los desarrolladores [4].

Una dimensión menos explorada del paradigma agéntico es la posibilidad de coordinar múltiples herramientas de LLM con roles diferenciados dentro de un mismo flujo de trabajo de desarrollo, aprovechando las fortalezas específicas de cada una. Este artículo informa sobre un estudio de caso exploratorio en el que se utilizaron tres herramientas: (1) Antigravity, un IDE

diseñado principalmente para agentes con un backend Gemini 2.5; (2) Gemini CLI, una interfaz de línea de comandos para modelos Gemini; y (3) Qwen Code, un modelo especializado en código ejecutado localmente. El estudio propone y aplica el Modelo de Especialización de Roles (RSM), un marco de coordinación que asigna un dominio de responsabilidad distinto a cada herramienta según sus capacidades.

El estudio se guía por tres preguntas de investigación: (RQ1) cómo se pueden coordinar herramientas basadas en LLM con diferentes capacidades a través del RSM en un flujo de trabajo de desarrollo real; (RQ2) qué desviaciones de la distribución de roles planificada emergen durante la ejecución del RSM y qué factores las explican; y (RQ3) cómo se compara el producto de software resultante con el modelo de calidad ISO/IEC 25010. Las contribuciones de este trabajo son: (a) el marco RSM para la coordinación de LLM multi-herramienta especializada por roles; (b) la documentación de un flujo de trabajo completo guiado por RSM y sus desviaciones; (c) la identificación de técnicas de blindaje de prompts observadas para respaldar la integridad de la salida de los agentes; (d) una evaluación cualitativa del producto bajo ISO/IEC 25010; y (e) un análisis de amenazas a la validez que respalda los esfuerzos de replicación.

El artículo se organiza de la siguiente manera: la Sección 2 revisa el trabajo relacionado; la Sección 3 describe el diseño del estudio de caso; la Sección 4 reporta la ejecución del estudio de caso; la Sección 5 presenta los resultados, la discusión, las implicaciones sociotécnicas, las amenazas a la validez y una validación teórica. Finalmente, la Sección 6 presenta las conclusiones y el trabajo futuro.

2. Trabajo Relacionado

2.1 De la Asistencia al Desarrollo Agéntico: El Marco SE 3.0

La trayectoria de la ingeniería de software se puede segmentar en etapas de abstracción incremental. Hassan et al. proponen un marco de autonomía de cinco niveles que caracteriza esta evolución (Tabla 1) [1]. El salto al Nivel 3 requiere, según los autores, una reestructuración de los pilares fundamentales de la disciplina: actores, procesos, herramientas y artefactos [1]. En este nuevo escenario, el ingeniero humano se convierte en un *Agent Coach* (Entrenador de Agentes), responsable de la especificación precisa de objetivos, la revisión del plan de ejecución y la auditoría de artefactos.

Li et al. [5] analizan la aparición de compañeros de equipo de IA (AI teammates) en el contexto de SE 3.0. Presentan AIDev, un conjunto de datos a gran escala que captura cómo los agentes de código autónomos colaboran con humanos en el desarrollo de software del mundo real. Su análisis muestra que los agentes de IA modernos son capaces de leer bases de código, planificar modificaciones, generar código y enviar solicitudes de extracción (pull requests), desplazando así el rol del desarrollador del mercado manual al nivel superior de orquestación y revisión.

2.2 El Fenómeno del Vibe Coding

El *vibe coding* fue introducido por Karpathy en febrero de 2025 en referencia a una modalidad de desarrollo en la que el programador se comunica con el modelo mediante lenguaje natural, casi sin escribir código directamente, aceptando los resultados de los agentes de manera relativamente pasiva [3]. La distinción académica relevante es: si el desarrollador revisa y comprende cada línea generada, utiliza un asistente; si acepta cambios basados en resultados superficiales, practica *vibe coding* en sentido estricto [3]. Sarkar y Drosos [6] presentan un estudio empírico del *vibe coding*, mostrando que sigue un modelo iterativo de satisfacción de objetivos, en el cual los desarrolladores alternan entre consultar a la IA, evaluar el código generado y realizar ediciones manuales. Sus resultados sugieren que el *vibe coding* no elimina la necesidad de experiencia en programación.

Esta práctica plantea inquietudes sobre la acumulación de deuda cognitiva y la erosión de habilidades técnicas fundamentales [4]. El presente estudio aborda este riesgo mediante la adopción de un modelo híbrido en el que el desarrollador humano actúa como un orquestador activo, revisando y aprobando los resultados antes de integrarlos al repositorio.

2.3 Generación de Código y Sistemas Multi-Agente

La generación de código basada en LLM ha avanzado sustancialmente desde que se introdujeron modelos como Codex [6] y AlphaCode [7]. Las pruebas de rendimiento estandarizadas como HumanEval [6] y MBPP [8] han demostrado que los modelos actuales resuelven una proporción significativa de problemas introductorios de programación; sin embargo, su rendimiento disminuye notablemente en tareas que requieren razonamiento sobre múltiples archivos o comprensión de arquitecturas de sistemas [9]. En el ámbito de la colaboración multi-agente, sistemas como MetaGPT [10] y ChatDev [11] asignan roles de equipos de software (analista, diseñador, programador, revisor) a instancias de LLM que se comunican entre sí, demostrando que la especialización de roles mejora la calidad del producto final. A diferencia de esos sistemas completamente automatizados, el presente trabajo explora un modelo híbrido con herramientas comerciales y de código abierto existentes coordinadas por un desarrollador humano.

2.4 Seguimiento de Instrucciones y Gestión de Contexto en Agentes LLM

La efectividad de los agentes de software está intrínsecamente ligada a su capacidad para procesar instrucciones complejas respetando restricciones críticas. Investigaciones recientes a través de la prueba de rendimiento AGENTIF han identificado un fenómeno de "degradación por densidad de restricciones", en el cual el rendimiento del agente disminuye a medida que aumenta el número de instrucciones simultáneas [12].

Un hallazgo particularmente relevante para este trabajo es el reportado por Li et al. [13]: el razonamiento explícito mediante Cadena de Pensamiento (CoT, por sus siglas en inglés, una técnica de prompting que instruye al modelo a razonar paso a paso antes de producir un resultado) puede, en ciertos casos, degradar la precisión en el seguimiento de instrucciones simples al redirigir la atención del modelo hacia procesos inferenciales complejos. Este efecto se discute en relación con una desviación observada en la Sección 5.2.

Respecto a la gestión del contexto del agente, un estudio reciente en ETH Zurich [14] demuestra que los archivos de reglas de proyecto demasiado detallados reducen las tasas de éxito en las tareas y aumentan los costos de inferencia. La recomendación derivada es adoptar instrucciones respaldadas por fallos (failure-backed instructions): agregar reglas solo cuando se haya demostrado que el agente comete un error recurrente sin ellas.

2.5 Evaluación de Herramientas de Asistencia al Desarrollo

Los estudios sobre el impacto de herramientas como GitHub Copilot muestran mejoras en la velocidad de escritura de código, pero también advierten sobre riesgos de sobreconfianza y una menor comprensión del código generado [15]. El presente trabajo contribuye a este cuerpo de conocimiento documentando la interacción entre múltiples herramientas agénticas a lo largo del ciclo de vida de desarrollo completo de un proyecto, un aspecto raras veces abordado en la literatura.

3. Diseño del Estudio de Caso

3.1 Tipo de Estudio y Método de Investigación

Este estudio está diseñado como un estudio de caso exploratorio siguiendo las directrices de Runeson y Höst [16] y Yin [17]. El caso es la aplicación del marco RSM a un proyecto real de desarrollo de software; la unidad de análisis es el RSM en sí mismo, examinado en términos de adherencia a roles, patrones de desviación y calidad del artefacto resultante. La recolección de datos consistió en la observación directa y la documentación sistemática de las interacciones con los agentes, prompts, salidas, desviaciones y resultados de pruebas durante la sesión de desarrollo. El análisis se realizó cualitativamente mediante la comparación de patrones con las tres preguntas de investigación definidas en la Sección 1. Este estudio no presenta un experimento controlado con variables independientes y dependientes; es una observación exploratoria orientada a caracterizar y documentar el comportamiento del RSM en un entorno de desarrollo natural, consistente con la categoría de "evaluación a pequeña escala" identificada por Wohlin y Rainer [18] cuando se estudia un fenómeno en un contexto único y no replicado.

3.2 Herramientas Empleadas

El experimento utilizó tres herramientas agénticas seleccionadas para representar distintos puntos en el espacio de diseño entre capacidad de razonamiento, privacidad y modalidad de uso.

Tabla 2. Distribución de roles entre herramientas en el Modelo de Especialización de Roles (RSM)

Herramienta	Rol Asignado	Modo de Ejecución	Fortaleza Principal
Antigravity	Arquitecto	IDE Agéntico (Nube)	Gestión de múltiples archivos, visión macro, refinamiento iterativo
Gemini CLI	Analista	Terminal (Nube)	Procesamiento masivo, generación de documentación y auditoría arquitectónica
Qwen Code	Especialista	Local (Ollama)	Validación de datos, pruebas unitarias, privacidad de datos sensibles

Antigravity es un IDE de código abierto derivado de la base de código de VS Code que adopta un enfoque agéntico prioritario [19]. Su característica distintiva es la integración nativa de un gestor de agentes capaz de ejecutar autónomamente tareas de múltiples pasos: crear archivos, navegar por el sistema de archivos, ejecutar pruebas y actualizar código. En la configuración utilizada, el backend fue el modelo Gemini 3 Flash de Google. Es importante señalar que, aunque algunos materiales de referencia mencionan un modelo "Gemini 3 Pro", esa versión no se utilizó en nuestro experimento; el modelo activo fue Gemini 3 en su variante Flash.

Gemini CLI es una herramienta de línea de comandos que permite la interacción con el modelo Gemini 3 desde la terminal del sistema en modo automático (Gemini CLI decide el mejor modelo para la tarea). Su fortaleza radica en procesar tareas que involucran múltiples archivos, generar texto extendido como documentación y automatizar flujos de trabajo mediante la redirección de salida en el sistema de archivos.

Qwen Code es un modelo de lenguaje especializado en programación desarrollado por Alibaba Cloud [20]. En este experimento, se ejecutó localmente a través de Ollama, garantizando la privacidad de los datos al evitar su transmisión a servidores externos.

3.3 El Modelo de Especialización de Roles (RSM)

Una contribución metodológica central de este trabajo es el Modelo de Especialización de Roles (RSM), inspirado en el principio de separación de intereses [21] aplicado no al código sino a las herramientas de asistencia. El RSM propone que cada herramienta asuma un dominio diferenciado de responsabilidad según sus capacidades particulares, de modo que las contribuciones sean complementarias y el solapamiento funcional se minimice. Los tres roles asignados se describen en la Tabla 2.

Esta distribución no es estática; como se documenta en la Sección 4.3, la ejecución real presentó desviaciones que se analizan en la Sección 5.2.

3.4 Proyecto del Estudio de Caso

Se diseñó un proyecto de complejidad moderada: una aplicación de escritorio en Python denominada *Climate Data Visualizer*, destinada a cargar archivos CSV con registros de temperatura y humedad y representarlos como gráficos de líneas interactivos. El proyecto se diseñó deliberadamente para involucrar múltiples capas técnicamente relevantes: diseño de interfaz gráfica de usuario (Tkinter), procesamiento de datos tabulares (Pandas), visualización (Matplotlib), validación de datos de entrada y pruebas unitarias automatizadas, de modo que cada rol del RSM tuviera un dominio de tareas no trivial a lo largo del ciclo de desarrollo.

3.5 Metodología de Evaluación de la Calidad

Para abordar la RQ3, el producto de software resultante se evaluó con respecto al modelo de calidad de producto ISO/IEC 25010:2011 [22]. Dada la naturaleza exploratoria de caso único del estudio, la evaluación se realizó como una evaluación cualitativa de expertos por parte del primer autor, un método consistente con estudios previos de caso único en calidad de software. Cada característica se operacionalizó de la siguiente manera: idoneidad funcional mediante la revisión de una lista de verificación de características frente a los requisitos originales; eficiencia de desempeño mediante la observación directa del tiempo de ejecución antes y después de la refactorización; confiabilidad mediante el monitoreo de ejecución y revisión de registros de errores; mantenibilidad mediante la inspección estática del código en cuanto a estructura de clases, acoplamiento y cobertura de anotaciones; capacidad de interacción a través de un recorrido por la interfaz; y flexibilidad mediante una prueba de ejecución en configuraciones cruzadas con y sin el entorno virtual. Las calificaciones (Alta / Moderada) reflejan juicios cualitativos frente a estas operacionalizaciones, no mediciones cuantitativas. Esta limitación se reconoce en la discusión de amenazas a la validez (Sección 5.6).

4. Ejecución del Estudio de Caso

4.1 Fase 1: Diseño Inicial del Software con Antigravity

El proceso de desarrollo comenzó con una instrucción en lenguaje natural al agente Antigravity, siguiendo un enfoque de prompting de cero disparos (zero-shot) [23], es decir, una descripción directa de la tarea sin ejemplos previos o directivas explícitas de cadena de pensamiento:

```
"Crea un proyecto en Python para visualizar datos climáticos desde un archivo CSV. Debe tener una interfaz gráfica básica usando Tkinter y permitir cargar el CSV y mostrar gráficos de líneas de temperatura y humedad a lo largo del tiempo. Asume que el CSV tendrá las columnas: fecha, temperatura, humedad."
```

En respuesta, Antigravity generó de manera autónoma la estructura de directorios, el archivo principal `climate_visualizer.py` implementando la interfaz con Tkinter y Matplotlib, scripts de soporte (`run.sh`, `requirements.txt`), documentación inicial (`README.md`, `QUICKSTART.md`) y un archivo de datos de muestra con 31 registros simulados. Un hallazgo relevante en esta etapa fue la tendencia del agente a generar código funcional pero monolítico: la clase `ClimateVisualizerApp` manejaba simultáneamente la lógica de la interfaz, la carga de datos, la validación y el renderizado del gráfico, violando el principio de separación de intereses [21]. Este diseño monolítico identificó la necesidad de una auditoría arquitectónica en fases posteriores.

4.2 Fase 2: Generación de Datos y Análisis con Gemini CLI

4.2.1 Blindaje de Prompts. La generación de un conjunto de datos de prueba de 100 registros mediante Gemini CLI reveló una vulnerabilidad crítica en el seguimiento de instrucciones: el agente intentó en sus dos primeras iteraciones invocar herramientas internas no disponibles en el entorno restringido de la terminal. Este comportamiento es consistente con el fenómeno de degradación por densidad de restricciones documentado en la prueba AGENTIF [12]. El prompt que produjo el resultado correcto incorporó restricciones negativas explícitas (Tabla 3), una técnica de blindaje de prompts esencial para prevenir comportamientos no deseados en agentes con acceso al sistema de archivos [24]:

```
gemini --prompt "Genera un archivo CSV con 100 líneas de datos climáticos simulados. NO ejecutes comandos. NO uses herramientas. NO intentes crear el archivo tú mismo. Simplemente escribe los datos como texto plano aquí en la terminal." > weather_data.csv
```

Tabla 3. Iteraciones de blindaje de prompts para la generación de CSV mediante Gemini CLI

#	Estrategia de Prompting	Resultado Técnico
1	Solicitud directa de generación de CSV	Error: el agente intentó invocar herramientas no disponibles
2	Instrucción de redirección de salida en terminal	Fallo parcial: mezcla de metadatos y texto plano en la salida
3	Inclusión de restricciones negativas explícitas	Éxito: generación limpia de datos redirigida al archivo

4.2.2 Auditoría Arquitectónica. A través de Gemini CLI en modo interactivo, se realizó un análisis arquitectónico del diseño inicial. El análisis identificó cuatro áreas de mejora: (a) violación del principio de separación de intereses [21]; (b) ineficiencia causada por la recreación del objeto `Figure` de Matplotlib en cada actualización; (c) ausencia de sugerencias de tipo (type hints); y (d) manejo excesivamente amplio de excepciones. Gemini CLI implementó las mejoras propuestas, introduciendo una clase `DataModel` para aislar la lógica de datos de la capa de presentación, consistente con una descomposición Modelo-Vista-Controlador.

4.3 Fase 3: Validación y Pruebas (Desviación del Plan Original)

Esta fase corresponde a la desviación más significativa con respecto al plan original del RSM. Bajo el RSM, a Qwen Code se le había asignado la tarea de refactorización; sin embargo, Gemini CLI asumió dicha tarea espontáneamente mientras ejecutaba el análisis arquitectónico de la Fase 2. Qwen Code se utilizó con un alcance más limitado: generar el módulo `validators.py` y la suite de pruebas unitarias.

El módulo `validators.py` verificó correctamente el formato de fecha AAAA-MM-DD, distinguiendo formatos válidos (p. ej., 2024-01-01) de los inválidos (p. ej., 2024/01/02 o 01-03-2024). La suite de pruebas generada por Qwen Code incluyó 10 casos de prueba en tres clases (`TestValidators`, `TestCSVValidation`, `TestDataModelBasic`), todos exitosos. Cabe señalar que la suite cubre únicamente el comportamiento a nivel unitario; se requerirían pruebas de integración y sistema antes de realizar afirmaciones sobre su despliegue, en conformidad con la práctica estándar de pruebas de software.

4.4 Fase 4: Interactividad Avanzada con Antigravity

El agente Antigravity implementó mejoras más allá de los requisitos originales: una barra lateral de estadísticas en tiempo real, información emergente (tooltips) interactiva al pasar el cursor sobre los puntos de datos, suavizado de media móvil configurable y exportación a PNG. Adicionalmente, añadió soporte para la carga de CSV desde la línea de comandos. Durante esta fase, el entorno virtual se corrompió; el agente se adaptó modificando `run.sh` para detectar la disponibilidad del entorno virtual y recurrir a la instalación de dependencias a nivel de sistema si era necesario.

4.5 Fase 5: Documentación Final con Gemini CLI

Gemini CLI consolidó `README.md` y `QUICKSTART.md` en un documento mejorado con una sección de inicio rápido y un historial de versiones de cinco hitos que registra la evolución del proyecto desde la implementación base hasta las características interactivas avanzadas.

5. Resultados y Discusión

5.1 Evaluación de la Calidad bajo ISO/IEC 25010 (RQ3)

Aplicando las operacionalizaciones definidas en la Sección 3.5, la Tabla 4 presenta la evaluación cualitativa del producto resultante con respecto a la norma ISO/IEC 25010:2011 [22].

Tabla 4. Evaluación cualitativa de la calidad del software bajo ISO/IEC 25010:2011.

Característica	Subcaracterística	Método de Evaluación	Resultado Observado
Adecuación funcional	Compleitud	Revisión de lista de verificación de características	Alta: todas las características planificadas fueron implementadas (gráficos, filtros, exportación, panel de estadísticas)
Eficiencia de desempeño	Comportamiento temporal	Observación del tiempo de ejecución	Mejorada tras refactorización: la reutilización de Figure/Canvas elimina la latencia al recargar datos
Confiabilidad	Ausencia de fallos	Revisión de ejecución y registro de errores	Moderada: sensible a la estabilidad del entorno de ejecución (se observó corrupción en venv)
Mantenibilidad	Modularidad / Analizabilidad	Inspección estática de código	Alta: estructura MVC, sugerencias de tipo en todo el código, constantes centralizadas de nombres de columnas
Capacidad de interacción	Compromiso del usuario	Recorrido por la interfaz	Alta: diseño limpio, barra de herramientas estándar de Matplotlib, tooltips interactivos
Flexibilidad	Adaptabilidad	Prueba en múltiples configuraciones	Alta: run.sh detecta el entorno automáticamente y aplica una instalación alternativa

La introducción de sugerencias de tipo y la creación de la clase `DataModel` fueron mejoras impulsadas por los agentes que aumentaron sustancialmente la analizabilidad y la capacidad de prueba del código. Sin embargo, la subcaracterística de confiabilidad se vio comprometida por la fragilidad del entorno de ejecución observada en la Fase 4, lo que subraya que la calidad del software agéntico no solo depende del código generado, sino también de la robustez del entorno de ejecución.

5.2 Análisis de Desviaciones con respecto al Plan Original (RQ2)

La principal desviación observada fue que Gemini CLI asumió la tarea de refactorización arquitectónica asignada originalmente a Qwen Code. Tres factores concurrentes explican esto: en primer lugar, la ausencia de un límite de alcance explícito entre la Fase 2 y la Fase 3 permitió a Gemini CLI implementar las mejoras propuestas sin una instrucción explícita para hacerlo. En segundo lugar, el solapamiento funcional entre ambas herramientas en tareas de refactorización desdibuja la frontera entre sus dominios. En tercer lugar, la inercia del flujo de trabajo: una vez que Gemini CLI había propuesto las mejoras, el costo cognitivo de cambiar de herramienta superaba el beneficio de mantener la distribución planificada. Específicamente, cambiar a Qwen Code en ese punto habría requerido reestablecer todo el contexto del proyecto en una herramienta diferente: transferir el contenido de los archivos relevantes, describir el trabajo ya completado y verificar que su resultado fuera consistente con lo que Gemini CLI ya había analizado. Este sobre costo de cambio de contexto, sumado al riesgo de producir resultados inconsistentes entre herramientas, constituyó el costo cognitivo que hizo de continuar con Gemini CLI la opción pragmática.

Esta observación es consistente con la literatura sobre sistemas multi-agente que señala la dificultad de mantener particiones estricta de responsabilidad cuando los agentes poseen capacidades solapadas [10]. Sugiere que, en flujos de trabajo reales, la distribución de roles entre agentes LLM tiende a adaptarse dinámicamente, recayendo en el orquestador humano la responsabilidad de intervenir activamente cuando la distribución planificada se considere importante.

Adicionalmente, el agente Antigravity ignoró inicialmente el formato de fecha AAAA-MM-DD HH:MM especificado en el prompt, lo que requirió una fase de validación posterior. Este comportamiento se puede explicar a posteriori mediante la paradoja del razonamiento CoT documentada por Li et al. [14]: cuando el agente se enfocó en la tarea de razonamiento estructural más amplia (generación del diseño de software), despriorizó una restricción de formato específica.

5.3 Efectividad Comparativa de las Herramientas (RQ1)

En función de la experiencia documentada, las fortalezas observadas de cada herramienta se pueden caracterizar de la siguiente manera: Antigravity generó seis archivos de proyecto en la Fase 1, demostrando una generación efectiva de múltiples archivos a partir de un solo prompt en lenguaje natural. Gemini CLI generó el conjunto de datos `weather_data.csv`, produjo el análisis arquitectónico que condujo a la refactorización de `DataModel` y consolidó la documentación en la Fase 5. Qwen Code generó el validador de formato de fecha y los diez casos de prueba unitarios dentro de su alcance definido de Especialista.

5.4 Desafíos Identificados

El experimento reveló tres desafíos recurrentes: la sensibilidad al prompt (la generación de CSV requirió tres iteraciones con restricciones negativas), la necesidad de verificación humana (aunque el código superó las pruebas unitarias, el desarrollador intervino varias veces) y la gestión explícita del contexto.

Tabla 5. Cambio en las métricas de valor profesional en la transición SE 1.0–2.0 → SE 3.0. Adaptado de [1].

Valores Tradicionales (SE 1.0/2.0)	Valores Agénticos (SE 3.0)
Escritura manual de algoritmos	Arquitectura y diseño del sistema
Depuración sintáctica	Orquestación de agentes y revisión de salidas
Velocidad de escritura/mecanografía	Claridad de especificación e intencionalidad
Pruebas unitarias manuales	Auditoría de planes de prueba agénticos

5.5 Implicaciones Sociotécnicas y Seguridad

5.5.1 Transformación del Rol del Desarrollador. La transición a SE 3.0 reconfigura el rol profesional del ingeniero de software. El valor del desarrollador se desplaza de la destreza sintáctica hacia la capacidad de descomponer problemas complejos estratégicamente y especificar intenciones con claridad (Tabla 5). Sin embargo, esto plantea inquietudes sobre el "arrastre de la IA" (AI drag) y la preservación del conocimiento técnico profundo en desarrolladores principiantes [4].

5.5.2 Patrones de Producción LLMOps. La estabilidad a largo plazo del software generado por agentes requiere adoptar prácticas de LLMOps: prompts como código (versionados en Git), separación entre planificación determinista y ejecución probabilística, y enrutamiento inteligente de modelos delegando tareas rutinarias a modelos locales pequeños.

5.5.3 Seguridad Agéntica. La autonomía otorgada a los agentes introduce nuevas superficies de ataque como las inyecciones indirectas de prompts y riesgos de integridad del entorno cuando el agente posee permisos de escritura en el sistema de archivos y acceso a la terminal [24].

5.6 Amenazas a la Validez

Se identifican amenazas a la validez interna (sesgo del observador al realizar el desarrollo y el análisis la misma persona), validez externa (estudio de caso único con herramientas de un solo proveedor en su mayoría), validez de constructo (evaluación cualitativa en lugar de métricas cuantitativas estrictas) y confiabilidad (naturaleza probabilística de los LLM).

5.7 Comparación con la Literatura Existente y Triangulación Teórica

El marco RSM se valida teóricamente demostrando su consistencia con tres cuerpos de conocimiento independientes: el principio de separación de intereses de Dijkstra [21], investigaciones en especialización de roles multi-agente (MetaGPT [10], ChatDev [11]) y el marco SE 3.0 de Hassan et al. [1].

6. Conclusiones y Trabajo Futuro

Este trabajo presentó un estudio de caso exploratorio del Modelo de Especialización de Roles (RSM). Se concluye que: (a) la especialización de roles es un principio de diseño válido para la coordinación de herramientas LLM pero requiere límites de alcance explícitos; (b) el blindaje de prompts mediante restricciones negativas explícitas es una técnica crítica; (c) la verificación humana sigue siendo indispensable; y (d) la paradoja del razonamiento CoT y la degradación por densidad de restricciones se observaron en condiciones reales de desarrollo.

El trabajo futuro contempla incorporar un cuarto agente evaluador bajo el paradigma LLM-as-a-Judge [26], evaluar el RSM con conjuntos de herramientas heterogéneos de diferentes proveedores, extender el estudio a proyectos de mayor escala y desarrollar métricas cuantitativas de efectividad de orquestación.

Referencias

- [1] Hassan, A.E.; Li, H.; Lin, D.; Adams, B.; Chen, T.-H.; Kashiwa, Y.; Qiu, D. Agentic Software Engineering: Foundational Pillars and a Research Roadmap 2025.
- [2] “Vibe Coding” Named Word of the Year by Collins Dictionary. BBC News, 2026.
- [3] Ray, P.P. A Review on Vibe Coding: Fundamentals, State-of-the-Art, Challenges and Future Directions. Authorea Prepr. 2025.
- [4] Osmani, A. Agentic Engineering 2026.
- [5] Li, H.; Zhang, H.; Hassan, A.E. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering 2025.
- [6] Chen, M.; et al. Evaluating Large Language Models Trained on Code 2021.
- [7] Li, Y.; et al. Competition-Level Code Generation with AlphaCode. Science 2022, 378, 1092–1097.
- [8] Austin, J.; et al. Program Synthesis with Large Language Models 2021.
- [9] Dong, Y.; et al. A Survey on Code Generation with LLM-Based Agents 2025.
- [10] Hong, S.; et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. ICLR 2023.
- [11] Qian, C.; et al. ChatDev: Communicative Agents for Software Development 2024.
- [12] Qi, Y.; et al. AGENTIF: Benchmarking Instruction Following of Large Language Models in Agentic Scenarios 2025.
- [13] Li, X.; et al. When Thinking Fails: The Pitfalls of Reasoning for Instruction-Following in LLMs.
- [14] Gloaguen, T.; et al. Evaluating AGENTS.Md: Are Repository-Level Context Files Helpful for Coding Agents? 2026.
- [15] Peng, S.; et al. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot 2023.
- [16] Runeson, P.; Höst, M. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. Empir. Softw. Eng. 2009, 14, 131–164.
- [17] Yin, R.K. Case Study Research: Design and Methods; Sage, 2009.
- [18] Wohlin, C.; Rainer, A. Is It a Case Study?—A Critical Analysis and Guidance. J. Syst. Softw. 2022.
- [19] Google. Introducing Google Antigravity, a New Era in AI-Assisted Software Development 2026.
- [20] Hui, B.; et al. Qwen2.5-Coder Technical Report 2024.
- [21] Dijkstra, E.W. On the Role of Scientific Thought. Springer 1982.
- [22] ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE).
- [23] Brown, T.; et al. Language Models Are Few-Shot Learners. NeurIPS 2020.
- [24] SPLX. The AI Agents That Trust Too Much: Are Your Agentic Workflows Vulnerable 2026.
- [25] ISO/IEC 25010:2023 Systems and software engineering — SQuaRE — Product Quality Model 2023.
- [26] Zheng, L.; et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023.